Modul: Einführung in die wissenschaftlich-technische Programmierung
SoSe 25/26

Praxis-Abschlussprojekt

Frankfurt University of Applied Sciences

FB02: Informatik und Ingenieurwissenschaften,

Mechatronik (B.Sc.), 2. Semester

Betreuer:innen: M.Sc. Virginia Kramer,

Jenish Lamichane (J)

Inhalt

1 Einleitung 3

2 Aufgabenstellung 3

3 Spielprinzip Mäxchen 4

4 Organisation 4

4.1 Aufteilung 4

4.2 Projektablaufplan 5

4.2.1 Phase 1: Konzeption und Planung 5

4.2.2 Phase 2: Hardware-Aufbau und erste Tests 5

4.2.3 Phase 3: Test, Entscheidung und Abschluss 6

5 Lösung 6

5.1 Led-Anzeige und Entwicklung 6

5.2 Software 8

5.3 Spielanleitung 11

6 Weitere Lösungsansätze 12

6.1 7-Segment-Display 12

6.2 8x8 Dot-Matrix-Displays mit MAX7219 13

6.3 Schlussfolgerung 14

7 Fazit 15

8 Quellenverzeichnis 16

1 Einleitung

Im Rahmen dieses Abschlussprojekts soll das klassische Würfelspiel „Mäxchen“ mithilfe eines Arduino-Mikrocontrollers und entsprechender Hardware realisiert werden. Der Spielablauf sowie die Auswertung erfolgen dabei über ein selbst erstelltes Programm in Simulink. Zur Darstellung der gewürfelten Augenpaare soll eine selbst entworfene elektronische Schaltung eingesetzt werden, die an den Mikrocontroller angeschlossen wird.

Ziel des Projekts ist es, die Funktionalität des Spiels technisch vollständig nachzubilden:

Zwei oder mehr Spieler würfeln nacheinander verdeckt mit zwei Würfeln. Nur der jeweils aktive Spieler kann das Ergebnis einsehen. Die Augenzahl wird über LEDs angezeigt, die wie ein Würfel angeordnet sind (s. Link zum Bausatz). Ein besonderes Merkmal des Spiels, das soge nannte Mäxchen (eine gewürfelte 1 und 2), soll vom System automatisch erkannt und durch eine spezielle visuelle Rückmeldung (z. B. Blinken aller LEDs) angezeigt werden.

2 Aufgabenstellung

Zur Umsetzung wird ein Arduino-Mikrocontroller verwendet (oder ein entsprechendes baugleiches Derivat), der über Simulink angesteuert wird. Simulink bietet eine geeignete Toolbox, um die Hardware ansteuern zu können. Wahrscheinlich ist es sinnvoll, die Stateflow-Toolbox für die vereinfachte Ablaufsteuerung in Simulink zu verwenden. Über die Hardware-Schnittstellen der Toolbox können sowohl Eingangssignale (Taster) eingelesen als auch Ausgänge (LEDs) angesteuert werden.

Für die Darstellung der Würfelergebnisse wird eine individuell gestaltbare elektronische Schaltung, z. B. auf einem Breadboard, entwickelt. Die Zufallszahlen zur Simulation der Würfe werden softwareseitig mit einem Zufallszahlengenerator (Random Number Generator) erzeugt. Um das verdeckte Würfeln zu ermöglichen, kann entweder ein Sichtschutz verwendet oder das Würfelergebnis erst nach Betätigen eines Buttons sichtbar gemacht werden.

Die konkrete Ausgestaltung von Software und Schaltung ist flexibel und kann frei gewählt werden.

Es gelten jedoch folgende Rahmenbedingungen:

- Die gewürfelten Augenpaare müssen über externe Hardware angezeigt werden.
- Das Ergebnis soll nur für den aktuellen Spieler sichtbar sein, muss aber bei Bedarf (der
- nächste Spieler will die gewürfelten Zahlen sehen) auch gezeigt werden können.
- Die Zufallszahlen werden per Software erzeugt.
- Ein Mäxchen muss automatisch erkannt und angezeigt werden.
- Wie schon erwähnt, sollen die LEDs wie auf einem Würfel angeordnet sein, da die An-
- zeige sehr viel intuitiver ist.

3 Spielprinzip Mäxchen

Mäxchen ist ein Würfelspiel für zwei oder mehr Spieler. Ziel ist es, eine höhere Zahl anzusagen als der vorherige Spieler. Der Spieler an der Reihe würfelt verdeckt, sieht sich das Ergebnis an und nennt dann eine Zahl – diese kann korrekt oder gelogen sein. Der nächste Spieler kann entscheiden, ob er die Ansage glaubt und selbst würfelt oder sie anzweifelt. Wird angezweifelt, wird der tatsächliche Wurf aufgedeckt – je nach Wahrheit oder Lüge gewinnt der entsprechende Spieler.

Die höhere Würfelzahl zählt als Zehner-, die niedrigere als Einerstelle. Pasche sind höher als normale Kombinationen. Das Mäxchen (1 und 2) ist die höchste Kombination und beendet das Spiel sofort.

4 Organisation

4.1 Aufteilung

Die Aufgabenverteilung sah vor, die geplanten Anzeigevarianten auf die einzelnen Aufgabe aufzuteilen. Einmal übernahm den Aufbau und die Steuerung der LED-Anzeige, der zwite methode konzentrierte sich auf die 7- Segmente-Anzeige und 8x8 dot matrix anzeige . Gleichzeitig unterstützte das zweite methode auch bei der Realisierung der LED-Anzeige und Schaltpläne. Durch diese Aufteilung war es uns möglich verschiedene Ansätze gleichzeitig auszuprobieren

4.2 Projektablaufplan

4.2.1 Phase 1: Konzeption und Planung

Zu Beginn des Projekts fand eine gemeinsame Besprechung statt, in der erste Ideen und Konzepte für die Umsetzung des Mäxchen-Spiels gesammelt wurden. Die ersten Gedanken über die Schaltung und die benötigten Komponenten wurden gemeinsam entwickelt und diskutiert. Es wurden verschiedene Anzeigemöglichkeiten wie einfache LEDs, 7-Segment-Displays und 8x8 Dot-Matrix-Displays in Betracht gezogen. Nach Abwägung der Vor- und Nachteile jeder Lösung wurden die grundlegenden Projektanforderungen und Spielregeln definiert. Die Aufgaben wurden so verteilt, dass paralleles Arbeiten möglich war: Eine Methode kümmerte sich um die Entwicklung der LED-Bargraph-Lösung, während die andere Methode die alternativen Anzeigelösungen (7-Segment und 8x8 Dot-Matrix) untersuchte.

Parallel zur Ideenfindung wurden die benötigten Hardware-Komponenten recherchiert und beschafft. Es wurden Informationen über die Ansteuerung von LEDs, die technischen Daten von 7-Segment-Displays (Typ 5461AS) und 8x8 Dot-Matrix-Displays mit MAX7219-Treiber eingeholt. Zeitgleich wurde die erste Version des Simulink-Modells entworfen und die grundlegende Struktur des Zustandsautomaten festgelegt.

4.2.2 Phase 2: Hardware-Aufbau und erste Tests

Mit dem Aufbau der LED-Schaltung auf dem Breadboard wurde begonnen. Zwölf LEDs wurden mit Vorwiderständen von 270 Ω versehen und an die digitalen Pins D2 bis D13 angeschlossen. Die vier Taster wurden an den analogen Pins A0 bis A3 angeschlossen und mit den internen Pull-up-Widerständen konfiguriert. Die unbeschalteten Pins A4 und A5 wurden als Antennen für den Zufallsgenerator vorgesehen.

Parallel dazu wurden erste Versuche mit den 7-Segment-Displays durchgeführt. Mit reinem Arduino-Code konnten identische Ziffern wie "1111", "5555" und "8888" erfolgreich dargestellt werden. Ebenfalls wurden erste Versuche zur Ansteuerung des 8x8 Dot-Matrix-Displays über den MAX7219-Treiber unternommen.

Die Spiellogik wurde in MATLAB Simulink modelliert. Es wurde ein endlicher Zustandsautomat mit drei Hauptzuständen (Setup, Game, Flashing) implementiert. Die Taster-Eingänge wurden mit NOT-

Gattern invertiert und mit Detect Rise Positive-Blöcken versehen, um das Prellen zu vermeiden. Ein Triggered Subsystem für den Würfelwurf wurde entwickelt, wobei die analogen Signale der unbeschalteten Pins A4 und A5 als Zufallsquelle dienten.

4.2.3 Phase 3: Test, Entscheidung und Abschluss

Nach Abschluss der Testphase wurde eine gemeinsame Entscheidung getroffen: Die LED-Bargraph-Lösung wurde als Hauptanzeige für das Projekt übernommen, da sie zuverlässig funktionierte und alle Anforderungen erfüllte. Die alternativen Ansätze mit 7-Segment- und 8x8-Displays wurden aufgrund der identifizierten technischen Hindernisse verworfen. Die Erkenntnisse aus diesen Versuchen wurden dokumentiert.

5 Lösung

5.1 Led-Anzeige und Entwicklung

Für die Darstellung der Würfelergebnisse wurde eine LED-basierte Anzeige entwickelt, bei der jeder Würfel durch sechs einzelne LEDs repräsentiert wird. Insgesamt kommen somit zwölf LEDs zum Einsatz. Diese sind direkt mit den digitalen Ausgängen des Arduino Uno verbunden.

Die LEDs des ersten Würfels (linke Seite) sind an die digitalen Pins D2 bis D7 angeschlossen, während die LEDs des zweiten Würfels (rechte Seite) die Pins D8 bis D13 verwenden. Jede LED ist über einen Vorwiderstand von 270 Ω mit dem jeweiligen Ausgang verbunden, um den Strom zu begrenzen und sowohl Mikrocontroller als auch LEDs vor Beschädigungen zu schützen. Alle LEDs teilen sich eine gemeinsame Masseleitung über die GND-Rail des Breadboards.

Im Gegensatz zu klassischen Würfeln, bei denen die Augen in einem 3 × 3 Raster angeordnet sind, wurde in diesem Projekt eine lineare Darstellung gewählt. Dabei entspricht die Anzahl der eingeschalteten LEDs direkt dem gewürfelten Wert. Dieses Verfahren wird auch als Bargraph-Darstellung bezeichnet. Ein Würfelergebnis von eins wird durch eine einzelne LED dargestellt, während bei einem Ergebnis von sechs alle sechs

LEDs aktiv sind.

Der Hauptgrund für diese Entscheidung liegt in der praktischen Umsetzung auf dem Breadboard. Eine lineare Anordnung der LEDs vereinfacht die Verdrahtung erheblich und reduziert mögliche Fehlerquellen während des Aufbaus. Zusätzlich bietet diese Darstellung den Vorteil, dass dieselben LEDs später auch zur Anzeige der verbleibenden Lebenspunkte der Spieler genutzt werden können. Dadurch lässt sich die Anzeige flexibel sowohl für die Würfelergebnisse als auch als Balkendiagramm für den Spielstand verwenden.

Die Steuerung des Spiels erfolgt über vier Taster, die an die analogen Pins A0 bis A3 angeschlossen sind. Um zusätzliche externe Bauteile zu vermeiden, wurden die internen Pull-up-Widerstände des Arduino aktiviert. Dadurch liegt im Ruhezustand ein HIGH-Signal am Eingang an. Wird ein Taster gedrückt, wird der entsprechende Pin auf LOW gezogen. Dieses Prinzip ermöglicht eine einfache und zuverlässige Auswertung der Tastensignale.

Zusätzlich werden die analogen Eingänge A4 und A5 als sogenannte Antenna Pins verwendet. Diese Pins bleiben absichtlich unbeschaltet und nehmen elektromagnetisches Umgebungsrauschen auf. Dieses analoge Rauschen wird anschließend in der Software genutzt, um Zufallszahlen für die Würfelwürfe zu erzeugen.

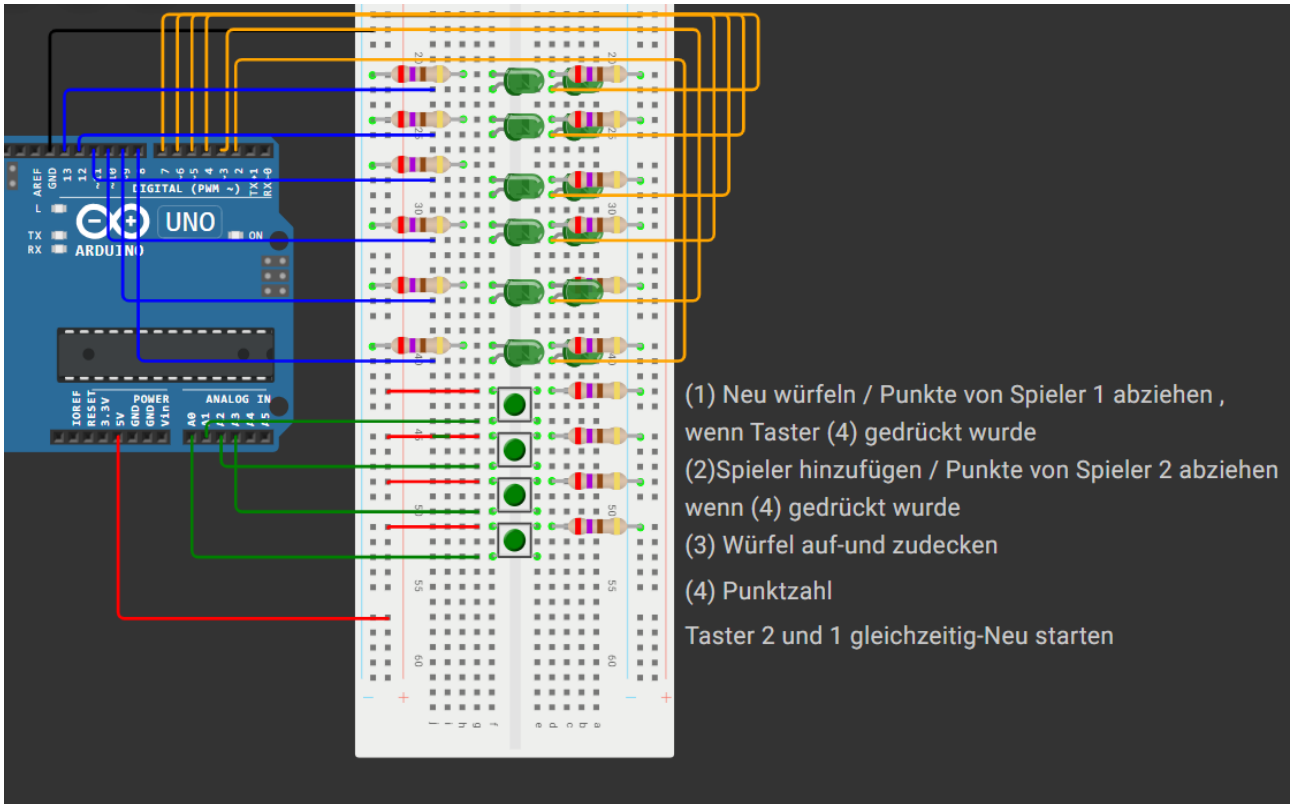


Abbildung 1: Aufbau der Schaltung

5.2 Software

Die Software des Projekts wurde vollständig in MATLAB Simulink modelliert. Der generierte Code kann direkt auf Arduino-Mikrocontroller übertragen werden [1][2]. Der von Simulink generierte C-Code wird anschließend direkt auf den Arduino Uno übertragen [1].

Das Simulink-Modell arbeitet mit einer festen Abtastrate von 0,1 Sekunden (10 Hz). Diese Samplingrate stellt sicher, dass Benutzereingaben zuverlässig erkannt werden. Gleichzeitig wird dadurch das mechanische Tastenprellen (Button Bouncing) effektiv reduziert.

Einlesen und Signalaufbereitung

Da die Taster mit aktivierten Pull-up-Widerständen arbeiten, liegt im Ruhezustand ein HIGH-Signal an. Beim Drücken eines Tasters entsteht ein LOW-Signal. Um diese Logik für die weitere Verarbeitung zu vereinfachen, werden die Eingangssignale zunächst durch NOT-Blöcke invertiert.

Für wichtige Ereignisse, wie beispielsweise das Auslösen eines Würfelwurfs, werden zusätzlich Detect Rise Positive-Blöcke eingesetzt. Diese Blöcke erkennen ausschließlich die steigende Flanke eines Signals. Dadurch wird sichergestellt, dass ein länger gedrückter Taster nur ein einziges Ereignis auslöst und nicht mehrfach hintereinander erkannt wird.

Zufallsgenerator und Triggered Subsystem

Das eigentliche Würfeln wird innerhalb eines Triggered Subsystems realisiert. Dieses Subsystem wird ausschließlich dann aktiviert, wenn eine steigende Flanke am Würfeltaster erkannt wird.

Innerhalb dieses Subsystems wird die MATLAB-Funktion rollTwoDice ausgeführt. Diese liest das analoge Signal der beiden unbeschalteten Pins A4 und A5 aus. Da diese Pins elektromagnetisches Umgebungsrauschen aufnehmen, liefern sie bei jeder Messung leicht unterschiedliche Werte.

Um aus diesen analogen Werten geeignete Zufallszahlen zu erzeugen, werden die Messwerte mit Primzahlen (173 und 137) multipliziert und anschließend über eine Modulo-6-Operation auf den Bereich 1 bis 6 begrenzt.

Anschließend wird ein Sortiermechanismus angewendet, der sicherstellt, dass der höhere Würfelwert immer auf dem linken Würfel dargestellt wird. Dieses Verhalten entspricht der klassischen Mäxchen-Regel, bei der die höhere Zahl zuerst genannt wird.

Da das Triggered Subsystem nur bei einem Tastendruck ausgeführt wird, bleibt das erzeugte Würfelergebnis auf den LEDs stabil angezeigt, bis ein neuer Wurf ausgelöst wird.

Spiellogik und Zustandsautomat

Die zentrale Steuerung des gesamten Spiels erfolgt innerhalb der MATLAB-Funktion gameLogic. Diese Funktion implementiert einen endlichen Zustandsautomaten (Finite State Machine) mit drei Hauptzuständen.

State 0 – Setup

Nach dem Einschalten oder einem Reset befindet sich das System im Setup-Modus. In

diesem Zustand zeigen die LEDs des linken Würfels die aktuelle Anzahl der Spieler an. Standardmäßig startet das Spiel mit zwei Spielern.

Mit Taster 2 kann die Spieleranzahl zyklisch von 2 bis 6 Spielern erhöht werden. Wird Taster 1 gedrückt, startet das Spiel und das System wechselt in den Spielmodus.

Zusätzlich verhindert ein Lockout-Timer, dass unmittelbar nach einem Reset versehentliche Mehrfachaktionen ausgelöst werden.

State 1 – Game

Im aktiven Spielmodus werden die Würfelergebnisse angezeigt. Durch Drücken von Taster 1 wird ein neuer Würfelwurf ausgelöst.

Über die Funktion hideToggle kann mit Taster 3 die Anzeige der LEDs vollständig ausgeblendet werden. Dadurch kann der Spieler das Ergebnis verdecken, ähnlich wie beim Becher im realen Mäxchen-Spiel.

Mit Taster 4 lässt sich der Score-Modus aktivieren. In diesem Modus zeigen die LEDs nicht mehr das Würfelergebnis, sondern die verbleibenden Lebenspunkte der Spieler an.

Während der Score-Modus aktiv ist, reduziert Taster 1 die Lebenspunkte von Spieler 1 und Taster 2 die Lebenspunkte von Spieler 2.

Sobald entweder ein Mäxchen (21) gewürfelt wird oder die Lebenspunkte eines Spielers auf null fallen, wechselt das System automatisch in den nächsten Zustand.

State 2 – Flashing

Dieser Zustand dient als visuelles Feedback für ein besonderes Ereignis, beispielsweise das Auftreten eines Mäxchens oder das Spielende.

Alle zwölf LEDs beginnen synchron zu blinken. Dieser Effekt wird über einen internen Zähler erzeugt, der das Blinken über etwa 40 Systemzyklen steuert. Nach Ablauf dieser Zeit

werden alle Spielvariablen zurückgesetzt und das System kehrt automatisch in den Setup-Modus zurück.

Zusätzlich kann das gesamte System jederzeit zurückgesetzt werden, indem Taster 1 und

Taster 2 gleichzeitig gedrückt werden. Diese Funktion wird durch die MATLAB-Funktion checkReset überwacht.

5.3 Spielanleitung

Voraussetzungen

Das System unterstützt 2 bis 6 Spieler. Die Bedienung erfolgt über vier Taster, während die visuelle Ausgabe über zwölf LEDs erfolgt.

Spielereingabe (Setup)

Nach dem Einschalten befindet sich das System im Setup-Modus. Die LEDs des linken Würfels zeigen die aktuelle Spieleranzahl an. Zwei leuchtende LEDs entsprechen beispielsweise zwei Spielern.

Durch Drücken von Taster 2 kann die Spieleranzahl bis maximal sechs erhöht werden. Mit Taster 1 wird die Auswahl bestätigt und das Spiel gestartet.

Spielablauf

Während des Spiels kann der aktive Spieler mit Taster 1 einen neuen Würfelwurf auslösen. Der linke Würfel zeigt stets den höheren Wert, während der rechte Würfel den niedrigeren Wert darstellt.

Mit Taster 3 kann die Anzeige der Würfel vollständig ausgeblendet werden. Dadurch kann das Gerät verdeckt weitergegeben werden, während der Spieler seine Aussage über das angebliche Würfelergebnis trifft. Ein erneuter Tastendruck zeigt das tatsächliche Ergebnis wieder an.

Durch Drücken von Taster 4 kann jederzeit in den Score-Modus gewechselt werden. In diesem Modus zeigen die LEDs des linken Würfels die verbleibenden Lebenspunkte von Spieler 1, während die LEDs des rechten Würfels die Lebenspunkte von Spieler 2 anzeigen. Solange sich das System im Score-Modus befindet, kann mit Taster 1 ein Lebenspunkt von Spieler 1 abgezogen werden. Taster 2 reduziert entsprechend die Lebenspunkte von Spieler 2. Durch erneutes Drücken von Taster 4 wird wieder zur normalen Würfelanzeige zurückgekehrt.

Mäxchen und Spielende

Die Würfelkombination 21 stellt im Spiel Mäxchen das höchste Ergebnis dar. Wird diese Kombination gewürfelt, erkennt das Programm dies automatisch.

Alternativ endet das Spiel auch dann, wenn im Score-Modus die Lebenspunkte eines Spielers auf null sinken. In beiden Fällen beginnen alle zwölf LEDs schnell zu blinken, um das Ereignis visuell zu signalisieren. Anschließend setzt sich das System automatisch zurück und kehrt in den Setup-Modus zurück.

6 Weitere Lösungsansätze

6.1 7-Segment-Display

Bei den Versuchen mit 7-Segment-Displays wurden zunächst grundlegende Tests durchgeführt. Es gelang, mit reinem Arduino-Code identische Ziffern auf mehreren Displays anzuzeigen. So konnten beispielsweise die Darstellungen "1111", "5555" oder "8888" problemlos realisiert werden.

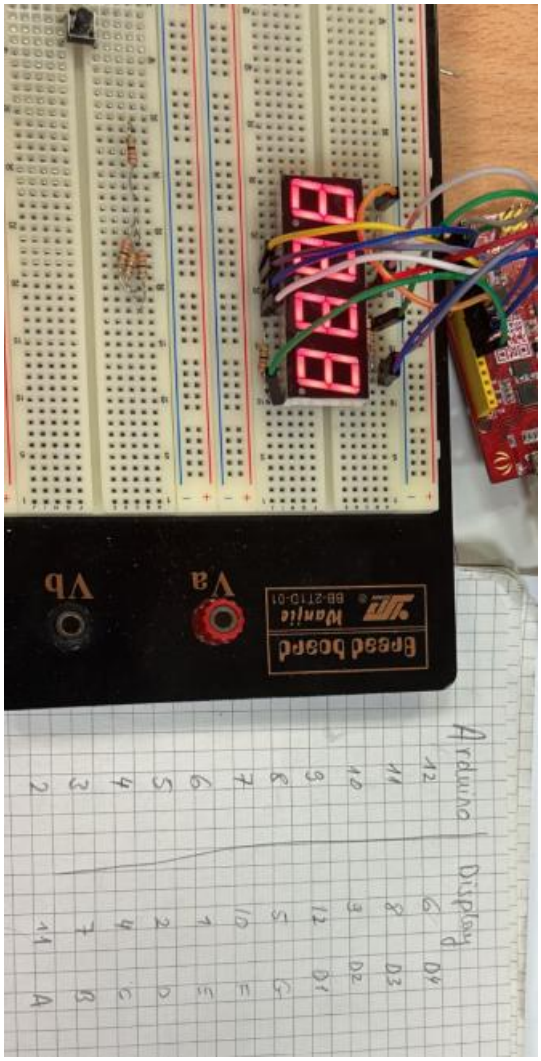
Die eigentliche Herausforderung zeigte sich bei dem Versuch, unterschiedliche Ziffern gleichzeitig darzustellen. Hierfür wäre Multiplexing zwingend erforderlich gewesen. Bei dieser Technik werden die Displays nacheinander so schnell angesteuert, dass das menschliche Auge den Eindruck einer gleichzeitigen Anzeige erhält.

Bei der Umsetzung gab es ein paar Probleme:

Die zeitliche Steuerung des Multiplexing war echt kritisch. Wenn man zu langsam umgeschaltet hat, sind die Displays sichtbar geflackert. Wenn man zu schnell umgeschaltet hat, wurden die Segmente nicht richtig angesteuert. Das hat dann zu fehlerhaften Anzeigen geführt.

Die Verwaltung der verschiedenen Zustände für jedes Display erforderte eine komplexe Pufferung. Jedes Display brauchte seinen eigenen Speicherbereich für die aktuell angezeigte Ziffer, und dieser Puffer musste in jeder Iteration neu geladen werden.

Die Integration in Simulink war echt der größte Brocken. In der blockbasierten Simulink-Umgebung hat das präzise Timing nicht so richtig funktioniert. Außerdem konnten wir die Arduino-spezifischen Funktionen, die wir für die direkte Hardware-Ansteuerung brauchten, in Simulink nicht nutzen.



6.2 8x8 Dot-Matrix-Displays mit MAX7219

Die Untersuchung von 8x8 Dot-Matrix-Displays konzentrierte sich auf die Ansteuerung über den MAX7219-Treiber-IC. Dieser ermöglicht prinzipiell die Kontrolle von 64 LEDs mit nur drei Datenleitungen.

In der Theorie bot diese Lösung Vorteile: Die Darstellung von Würfelaugen wäre direkt möglich gewesen, ohne den Umweg über Ziffern. Die Ansteuerung über SPI-ähnliche Kommunikation erschien zunächst elegant.

In der Praxis zeigten sich jedoch erhebliche Probleme:

Die Kommunikation mit dem MAX7219 erfolgt über die Arduino-spezifischen Funktionen shiftOut und digitalWrite. Diese Funktionen sind Teil der Arduino-Standardbibliothek und setzen

die Einbindung von Arduino.h voraus. Bei dem Versuch, diese Funktionen aus Simulink heraus aufzurufen, trat wiederholt der Fehler auf, dass die Datei Arduino.h nicht gefunden werden konnte.

Der Build-Prozess scheiterte regelmäßig mit der Meldung:

text

fatal error C1083: Datei (Include) kann nicht geöffnet werden: "Arduino.h"

Dieser Fehler verdeutlicht die grundsätzliche Inkompatibilität: Simulink verwendet einen eigenen C-Compiler (Visual Studio) für die Codegenerierung, der nicht auf die Arduino-spezifischen Bibliotheken zugreifen kann.

Es wurden mehrfach Versuche unternommen, dieses Problem mit den in Simulink verfügbaren SPI-Blöcken zu umgehen. Diese Blöcke sind zwar für allgemeine SPI-Kommunikation vorgesehen, erwiesen sich jedoch für die spezifische MAX7219-Ansteuerung als ungeeignet. Die Konfiguration der SPI-Parameter wie Clock-Polarität, Datenformat und Timing ließ sich nicht so einstellen, dass eine zuverlässige Kommunikation mit dem MAX7219 zustande kam.

Auch der Einsatz von anderen Kommunikationsblöcken aus der Simulink-Bibliothek führte zu keinem Erfolg. Die Blöcke sind für standardisierte Kommunikationsprotokolle ausgelegt, nicht für die spezifischen Anforderungen der MAX7219-Ansteuerung. Die Versuche, mit diesen Blöcken eine funktionierende Verbindung aufzubauen, blieben letztlich belanglos und führten zu keiner praktikablen Lösung.

Die zeitlichen Anforderungen für die MAX7219-Kommunikation verschärften die Problematik weiter. Die Signale müssen mit präzisen Zeitsteuerungen gesendet werden, was in Simulink kaum zu realisieren ist.

6.3 Schlussfolgerung

Die vergleichende Untersuchung der verschiedenen Anzeigelösungen führt zu folgenden Erkenntnissen:

Die 7-Segment-Displays erwiesen sich trotz erster Erfolge bei identischen Ziffern als ungeeignet für die Anforderungen des Mäxchen-Spiels. Die Komplexität des Multiplexing für unterschiedliche Ziffern und die Schwierigkeiten bei der Simulink-Integration machten eine zuverlässige Umsetzung unmöglich.

Die 8x8 Dot-Matrix-Displays mit MAX7219 boten zwar theoretisch bessere Darstellungsmöglichkeiten, scheiterten aber an der grundsätzlichen Inkompatibilität zwischen Simulink und den Arduino-

Bibliotheken. Die wiederkehrenden Build-Fehler beim Einbinden von Arduino.h demonstrierten dieses Problem eindrücklic

7 Fazit

7.1 Fazit

Die Entwicklung dieses Projekts ermöglichte einen tiefen Einblick in die Arbeit mit MATLAB Simulink und die Herausforderungen bei der hardwarenahen Programmierung. Besonders lehrreich war die Erkenntnis, dass nicht immer die technisch ausgefeilteste Lösung die praktikabelste ist. Die Entscheidung für die einfache LED-Bargraph-Darstellung anstelle komplexerer Display-Lösungen ermöglichte erst die termingerechte und zuverlässige Fertigstellung des Projekts.

Die Arbeit mit Simulink als Modellierungsumgebung für eingebettete Systeme hat die Vorteile der blockbasierten Programmierung, aber auch deren Grenzen bei hardwarenahen Anwendungen aufgezeigt. Die Kombination aus Simulink-Codegenerierung und Arduino-Hardware bietet großes Potenzial für zukünftige Projekte, erfordert jedoch eine sorgfältige Abwägung der Kompatibilität von Komponenten.

Das Projektziel, ein funktionsfähiges Mäxchen-Spiel mit Arduino und Simulink zu entwickeln, wurde erreicht. Das System arbeitet zuverlässig, bildet alle wesentlichen Spielelemente ab und bietet eine intuitive Benutzerschnittstelle. Die Dokumentation der technischen Herausforderungen und der Vergleich mit alternativen Lösungsansätzen liefert wertvolle Erkenntnisse für ähnliche zukünftige Projekte.

8 Quellenverzeichnis

[1] MathWorks: Arduino Support Package für MATLAB und Simulink.

Verfügbar unter: https://de.mathworks.com/hardware-support/arduino.html

Zugriff am: 2025.

[2] MathWorks: Arduino Programming with MATLAB and Simulink.

Verfügbar unter: https://de.mathworks.com/discovery/arduino-programming-matlab-simulink.html

Zugriff am: 2025.

[3] Claude AI (Anthropic): Unterstützung bei der Ideenfindung, Fehlersuche und Code-Diskussion während der Entwicklung des Projekts. Die endgültige Implementierung, Tests und Anpassungen wurden eigenständig durchgeführt.

[4] Maxim Integrated Products: *MAX7219/MAX7221 Serially Interfaced, 8-Digit LED Display Drivers Datasheet*. Rev 4; 7/03. Verfügbar unter: https://datasheets.maximintegrated.com/en/ds/MAX7219-MAX7221.pdf

[5] Toplight Technology Co., Ltd.: *CC-5461AS - 0.56 Inch (14.2mm) Four Digit Segment Display Datasheet*. Verfügbar unter: http://www.ledtoplight.com.cn/FileUpload/CC-5461AS.pdf (Zugriff am: 17.03.2025)